Wystąpienie 3 - aktualny postęp prac

Projekt i implementacja aplikacji webowej do analizy danych dotyczących snu i dziennego samopoczucia użytkownika z generowaniem rekomendacji

Maciej Zadrożny

Promotor: dr inż. Tomasz Łukaszuk

Problem i cel

Dane o śnie i samopoczuciu istnieją, ale rzadko są łączone w jedną, zrozumiałą całość.

Problem

- użytkownik nie wie, które nawyki (kawa, alkohol, ekrany, stres) realnie pogarszają jego sen,
- zmęczenie odczuwane rano nie wskazuje przyczyny gorszego dnia,
- istniejące narzędzia obejmują tylko fragment problemu.

Cel pracy

Aplikacja webowa łącząca raport poranny i wieczorny w jeden dzienny log, wyliczająca DayScore, statystyki oraz rekomendacje o charakterze informacyjnym (nie diagnostycznym).

Przegląd istniejących rozwiązań (rozdz. 2 pracy)

Kalkulatory snu

uniwersalny wzór 90 min, brak historii i danych o samopoczuciu

Sleep as Android

szczegółowa analiza nocy z sensorów, słabe powiązanie z oceną dnia; część funkcji płatna

Daylio

nastrój i aktywności, brak szczegółów snu (godziny, wybudzenia, realny czas snu)

Najważniejsze wymagania

Wymagania funkcjonalne opisują ścieżkę użytkownika, niefunkcjonalne - jakość i bezpieczeństwo danych.

Funkcjonalne

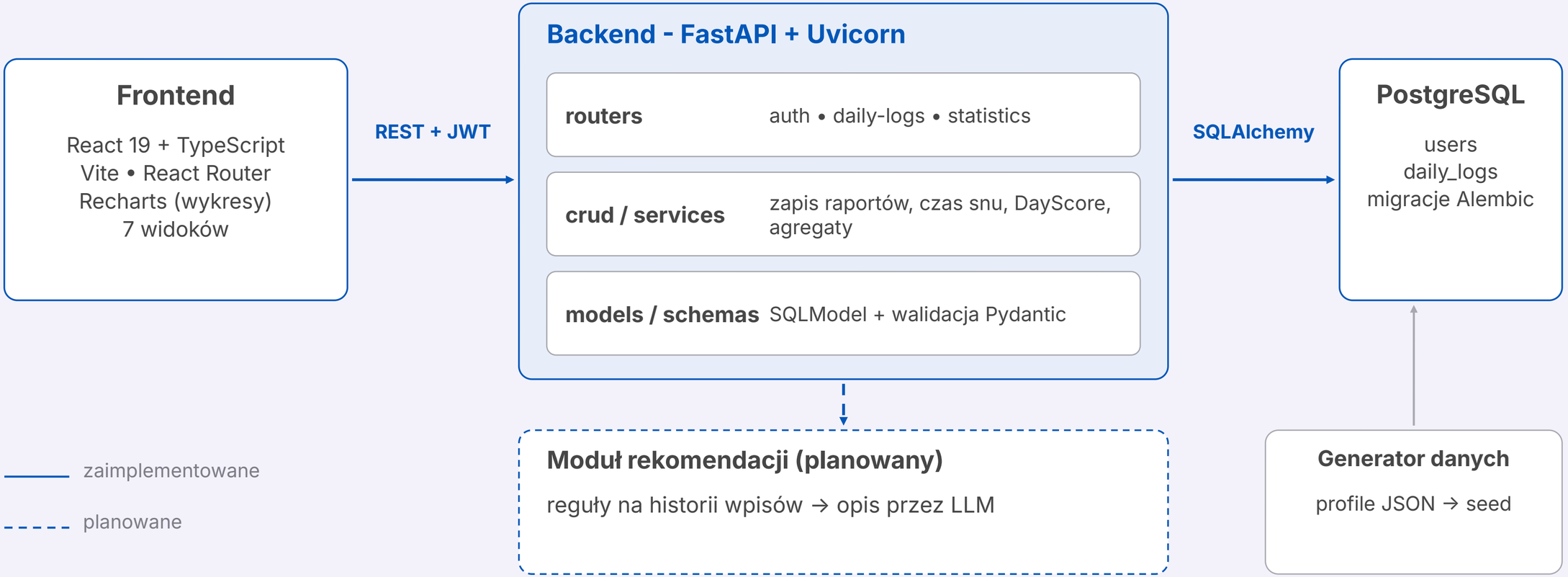
- **F1** Rejestracja i logowanie (token JWT)
- **F2** Raport poranny: godziny snu, jakość, wybudzenia, energia
- **F3** Raport wieczorny: ocena dnia, stres, kawa, alkohol, ekrany, drzemka
- **F4** Historia wpisów z podglądem i edycją
- **F5** Statystyki i wykresy dla wybranego zakresu dat
- **F6** Rekomendacje na podstawie historii użytkownika

Niefunkcjonalne

- **N1** Bezpieczeństwo: hasła bcrypt, dostęp tylko do własnych wpisów
- **N2** Poprawność danych: walidacja zakresów ocen i czasu w łóżku (maks. 18 h)
- **N3** Szybki wpis: krótkie formularze, wartości kategoryczne
- **N4** Rozszerzalność: warstwowy backend, migracje Alembic
- **N5** Charakter informacyjny, bez funkcji diagnostycznych

Architektura rozwiązania

Klient-serwer: frontend SPA, warstwowe REST API w FastAPI i relacyjna baza danych.



Model danych i przepływ raportów

Jeden rekord DailyLog opisuje noc oraz dzień, który po niej nastąpił.

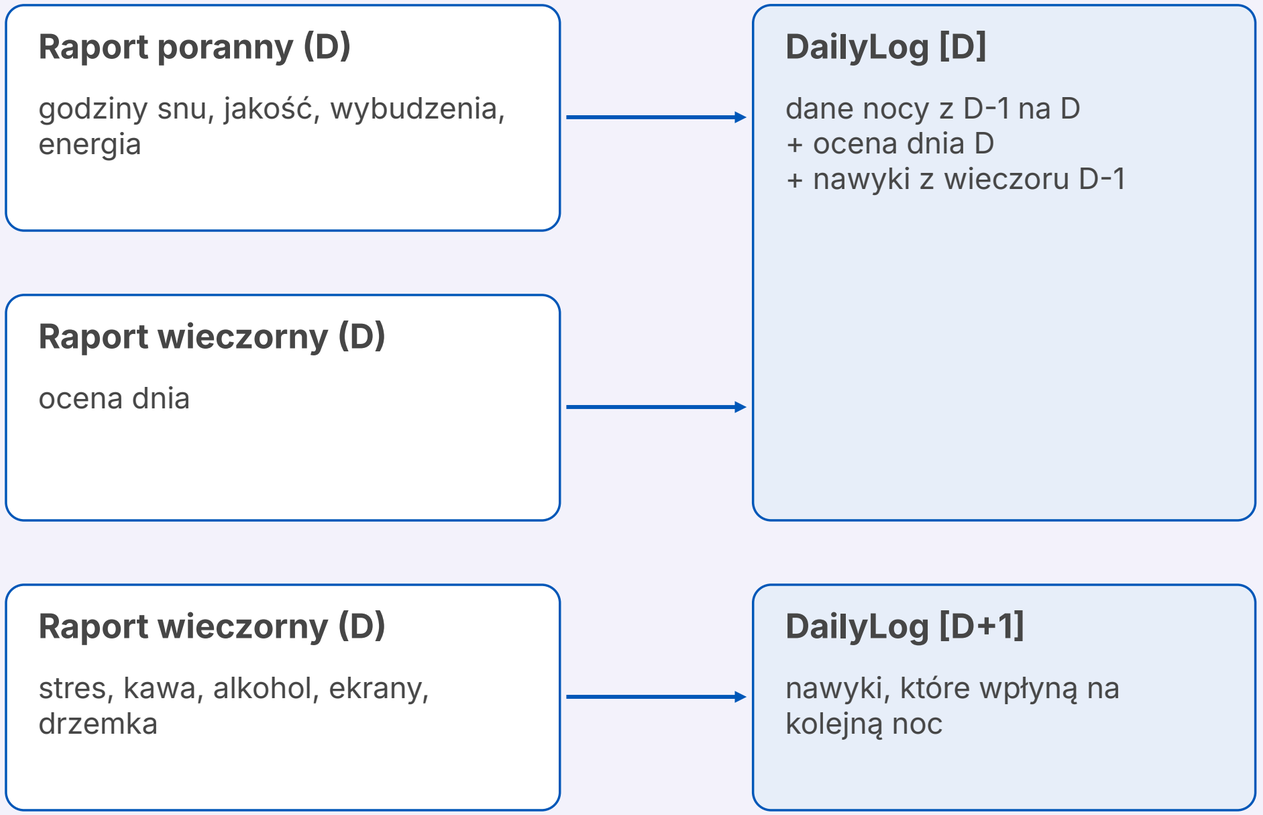


Tabela daily_logs

Identyfikacja	user_id → users, date
Rano	sleep_start, sleep_end, sleep_latency_extra, sleep_quality, night_awakenings, morning_energy
Wieczór	day_rating, stress_level, coffee_last_6h, alcohol_last_4h, screens_last_hour, nap_type
Wyliczane	sleep_duration, day_score

users: e-mail, hasło (hash), imię, płeć, wiek, typowy czas zasypiania • relacja 1:N

Decyzje projektowe: dane i obliczenia

Każda decyzja upraszcza późniejszą analizę zależności między nawykami a snem.

1 Jeden rekord na dzień (upsert)

Wpis identyfikowany przez użytkownika i datę. Raporty mogą przyjść w dowolnej kolejności lub tylko jeden z nich, a statystyki to proste zapytanie po zakresie dat.

2 Nawyki wieczorne w rekordzie D+1

Kawa, alkohol czy ekrany wpływają na noc opisywaną następnego ranka, więc przyczyna i skutek leżą w jednym wierszu.

3 Realny czas snu z profilu

Czas w łóżku minus typowy czas zasypiania z profilu i dodatkowe opóźnienie. Użytkownik nie musi znać dokładnej godziny zaśnięcia.

4 Wskaźnik DayScore

DayScore =

0,25 · jakość snu
+ 0,25 · energia rano
+ 0,5 · ocena dnia

Połowa wyniku to całościowa ocena dnia, druga połowa opisuje regenerację po nocy. Skala 1-10, wartość zapisywana przy każdym zapisie i edycji wpisu.

Decyzje projektowe: testy i rekomendacje

Wybory przygotowujące system do modułu rekomendacji.

	Decyzja	Uzasadnienie
5	**Nawyki jako wartości kategoryczne** kawa i alkohol: tak/nie w oknie 6 h / 4 h przed snem, ekrany: low/medium/high, drzemka: none/short/medium/long	szybki wieczorny wpis, porównywalne dane, proste reguły (dni z kawą vs bez)
6	**Dane syntetyczne z profili JSON** 5 osób z okresami specjalnymi (sesja, urlop, deadline) i stałym seedem	brak realnych użytkowników na tym etapie, powtarzalne dane ze znanymi zależnościami
7	**Rekomendacje: reguły + LLM (kierunek wstępny)** reguły liczą fakty z historii, LLM opisuje je językiem naturalnym; ML tylko opcjonalnie	mniejsze ryzyko odpowiedzi niezgodnych z danymi, niezależność od jednego dostawcy
8	**Warstwowy backend i bezstanowe JWT** routers / crud / services / schemas, walidacja w schematach Pydantic	logika statystyk i rekomendacji rozwijana niezależnie od API

Stan implementacji

Pełna ścieżka użytkownika działa: od konta, przez raporty, po historię i statystyki.

Uwierzytelnianie: rejestracja, logowanie, JWT	gotowe
Raporty poranny i wieczorny (logika D / D+1)	gotowe
Czas snu i DayScore	gotowe
Historia wpisów: podgląd i edycja	gotowe
Statystyki: API, średnie, wykresy	**rozwijane**
Generator danych syntetycznych + seeder	gotowe
Panel główny i dopracowanie interfejsu	**w trakcie**
Moduł rekomendacji	**kolejny etap**
Testy automatyczne i scenariuszowe	**planowane**

9

endpointów REST (auth, daily-logs, statistics)

7

widoków aplikacji (React + Recharts)

~4,7 tys.

wpisów syntetycznych (5 osób × ~940 dni)

3

dokumenty pracy pisemnej (konspekt, wstęp, przegląd rozwiązań)

Nowe moduły: historia i statystyki

Od poprzedniego wystąpienia aplikacja nie tylko zapisuje dane, ale też pozwala je przeglądać i analizować.

Historia wpisów

GET /daily-logs

lista wpisów z paginacją (limit, offset)

↓

GET /daily-logs/{id}

podgląd raportu porannego i wieczornego

↓

PATCH /daily-logs/{id}

częściowa edycja, ponowna walidacja i przeliczenie

Użytkownik może pobrać i edytować wyłącznie własne wpisy, a czas snu i DayScore przeliczają się przy każdej zmianie.

Panel statystyk

GET /statistics?date_from&date_to

wybór okresu i liczba raportów

↓

summary

średnie: sen, jakość, energia, wybudzenia, ocena dnia, stres

↓

daily → Recharts

punkty dzienne do wykresów długości i jakości snu

Moduł statystyk jest bazą pod rekomendacje: te same agregaty posłużą regułom.

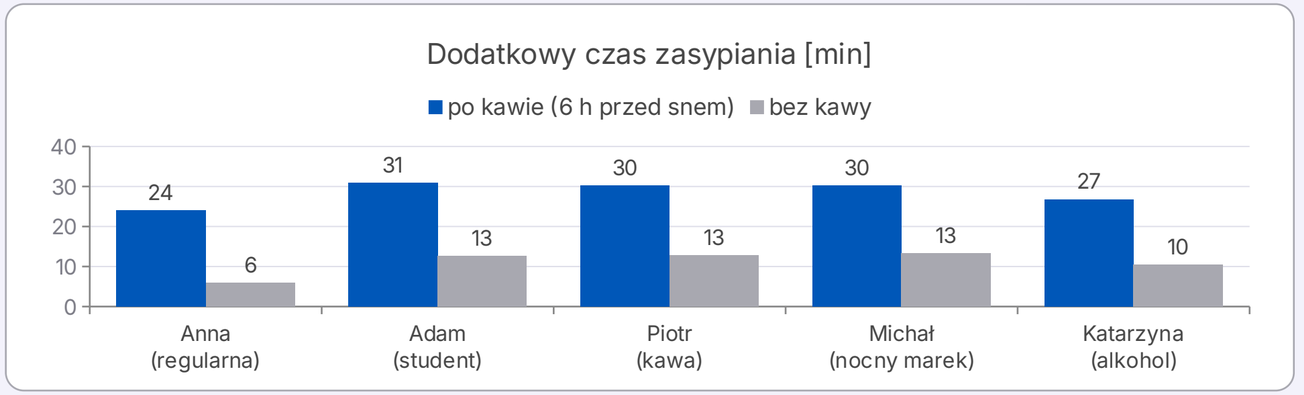
Dane syntetyczne do testów i walidacji

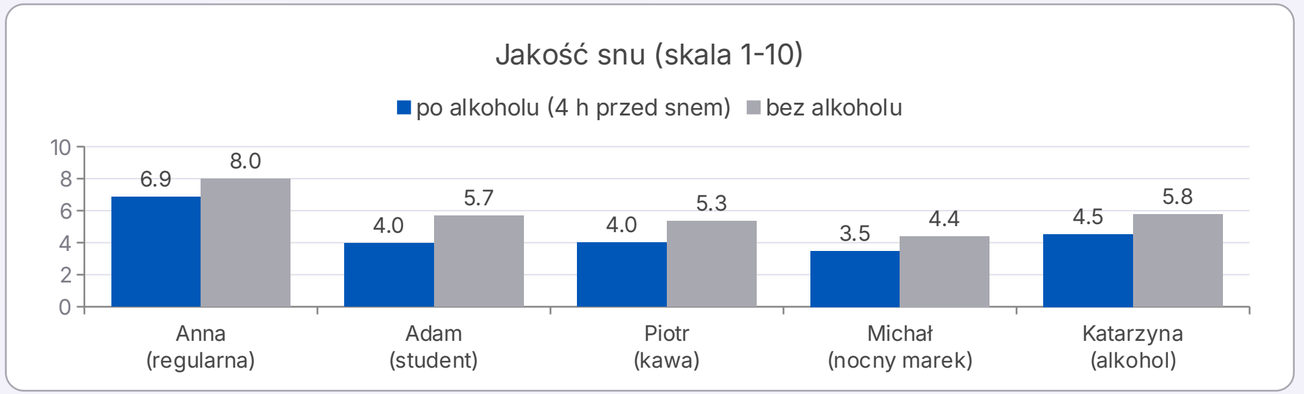
Generator odtwarza ok. 2,5 roku historii (01.2024 - 08.2026) dla każdej z 5 osób.

Jak działa generator

- profil JSON: wartości bazowe snu i ocen, prawdopodobieństwa nawyków
- weekendy i okresy specjalne: sesja, urlop, deadline
- wieczorne nawyki wpływają na kolejną noc
- ok. 90% wypełnionych raportów, stały seed

Zależności są celowo wbudowane, więc moduł reguł powinien je wykryć. To gotowy test poprawności rekomendacji.





Problemy i ryzyka

Każde ryzyko ma przypisane konkretne działanie ograniczające.

1 Ograniczony czas

praca zawodowa równolegle z ostatnim etapem studiów i pisaniem pracy

Działanie

podział zakresu na część obowiązkową (reguły + LLM, testy) i opcjonalną (ML, rozbudowany interfejs)

2 Mało realnych danych

brak grupy użytkowników, a dane od jednej osoby nie wystarczą do oceny rekomendacji

Działanie

dane syntetyczne ze znanymi zależnościami; moduł ML tylko jako eksperyment

3 Otwarta decyzja o LLM

wybór modelu i dostawcy: koszt, prywatność danych, jakość odpowiedzi po polsku

Działanie

najpierw moduł reguł niezależny od dostawcy, LLM jako wymienna warstwa opisowa

Plan dalszych prac

Najbliższy cel: działające rekomendacje na danych syntetycznych.

XI • teraz

Moduł reguł

- agregaty: kawa → zasypianie, alkohol → jakość i wybudzenia, ekrany, stres
- porównanie tygodnia i weekendu
- decyzja co do modelu LLM
- endpoint /recommendations

XII

LLM, interfejs, testy

- integracja LLM i widok rekomendacji
- statystyki nawyków, trend DayScore, panel główny
- testy pytest i scenariuszowe
- pisanie rozdziałów 3-6

I

Ewaluacja i finalizacja

- ocena zgodności rekomendacji z danymi syntetycznymi
- rozdziały 7-9
- poprawki, przygotowanie do obrony

Dziękuję za uwagę